

CEN455

Secure Smart-Home IoT Communication

Talal Hajjaj

Student ID: 1086598

Dema Abdullah

Student ID: 1084800

Instructor: Dr. Mohamed Fadul

I. INTRODUCTION

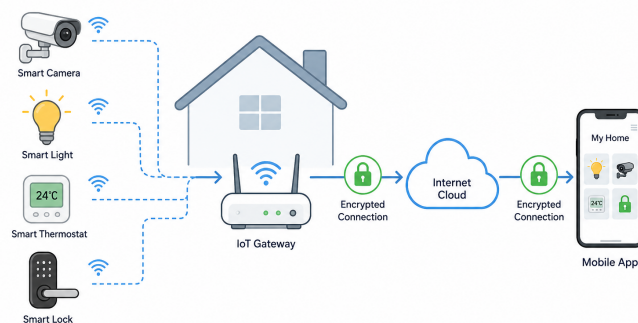
Rapid evolution of the Internet of Things (IoT) has converted typical dwellings into smarter living spaces in which interconnected devices interact seamlessly to enhance comfort, efficiency, and security. Smart homes make use of sensors, actuators, embedded devices, cloud computing solutions, and wireless communication technology to automate routine tasks like lighting automation, environment monitoring, home security, and energy management. As per the latest market statistics, there are billions of IoT devices that have already been installed across the world, and one of the most rapidly growing industries is the industry of smart homes. Smart home solutions are usually built on inexpensive embedded devices like ESP32, Raspberry Pi, and microcontrollers based on the ARM processor architecture. Such devices can be connected wirelessly while ensuring energy savings. The connection among these devices can be established using lightweight messaging protocols, like the Message Queuing Telemetry Transport (MQTT), which uses the publisher-subscriber architecture. Using MQTT reduces the overhead of communication, making it the best choice for limited IoT devices communicating wirelessly. However, MQTT was initially intended to be efficient but not secure, which makes smart homes open to cyberattacks without proper security protocols. With an increase in the use of smart-home technologies, the attack surface for cybercriminals has increased as well. Hackers may access the smart-devices of users without their consent and use them to violate users' privacy or interfere with other functionalities. Therefore, secure communications in IoT devices have become one of the major research areas in embedded systems.

A. Objectives of the Literature Review

This literature review focuses on the latest achievements in protecting IoT communications in the smart-home environment, especially those that use the MQTT protocol in embedded systems. In this research paper, we would like to find out what kinds of cyberattacks pose the greatest threat to IoT environment, what security mechanisms are offered by scientists, and what lightweight cryptography can be applied in such devices. We pay special attention to authenticated encryption that uses AES-GCM, secure communication using MQTT

protocol, and post-quantum cryptography using ML-KEM-512 (Kyber) algorithm, which has recently been standardised by NIST. In addition, we analyze the possibility of applying these security measures on ESP32 chips, analyzing their execution time, memory usage, and overhead of communication. The results of this literature review will form the theoretical background for further stages of our project, where we will try to develop a secure MQTT communication environment for smart home that will resist such types of attacks as eavesdropping, spoofing, replay, and message tampering.

Secure Smart-Home IoT Communication



II. SMART-HOME AND IOT SECURITY THREATS

A. Overview of IoT Threat Landscape

The Internet of Things (IoT) has changed conventional domestic environments into interconnected smart homes that can offer automation, remote control, and intelligent decisions. The IoT technologies that have become integrated into the smart home include smart locks, light systems, cameras for surveillance, motion sensors, and monitoring of the environment that constantly exchange data via wireless networks. Although IoT has contributed a lot in terms of increased convenience and efficiency, it poses a number of cybersecurity issues since the embedded devices have restricted computational capabilities. Unlike computer networks, IoT environments comprise heterogeneous devices with different hardware, communication protocols, and operating systems. Many IoT devices are developed with restricted memory capacity and restricted processing

power and energy consumption that prevent the use of proper security measures. Consequently, the manufacturers focus on functionality and cost-effectiveness rather than cybersecurity.

Most researchers tend to group IoT attacks into either passive or active attacks. The passive attacks aim at collecting confidential information without changing the flow of the network traffic, while the active attacks change the communications themselves. Some of the attacks that have been reported in the literature include:

- Eavesdropping
- Spoofing
- Replay Attack
- Message Modification
- Denial of Service Attack
- Malware and botnet attack
- Physical attack on embedded devices

The knowledge about these threats is vital when designing IoT communication system for security purposes.

TABLE I
COMMON SECURITY ATTACKS IN SMART HOME IoT SYSTEMS

Attack	Description	Impact
Eavesdropping	Intercepts network traffic.	Privacy leakage and sensor data exposure.
Spoofing	Impersonates legitimate IoT devices.	Unauthorized appliance control.
Replay	Reuses captured messages.	Executes outdated valid commands.
Message Tampering	Modifies transmitted messages.	False readings and incorrect operation.
DoS	Floods devices or brokers with traffic.	Communication failure and service disruption.
Malware/Botnets	Infects IoT devices.	Device takeover and distributed attacks.
Physical Attacks	Gains direct hardware access.	Key extraction and firmware modification.

III. MQTT SECURITY AND PROTECTION MECHANISMS

A. MQTT Protocol Overview

Message Queuing Telemetry Transport (MQTT) is a lightweight messaging protocol developed for facilitating communications between resource-limited devices via unreliable or bandwidth-constrained network connections. The technology was initially created by IBM in 1999 but has evolved into one of the most popular communication protocols used in Internet of Things (IoT) systems because of its simplicity, scalability, and low communication overhead. Instead of the client-server model that requires devices to communicate directly with each other, the MQTT protocol uses the publish-subscribe pattern. This means that all communications between devices happen through an MQTT broker server. While devices act as publishers and post messages on particular topics, subscribers receive messages only from topics they have subscribed to. As applied to smart home systems, MQTT technology is used to enable communication between sensors, actuators, embedded controllers, and cloud services. For instance, an ESP32 door sensor can publish the state to the following topic:

home/frontdoor/status

A second ESP32 managing an electronic lock will subscribe to this topic and execute the respective action after receiving the message. The MQTT protocol provides three Quality of Service (QoS) levels:

- QoS 0 - At most once: Messages are delivered without any acknowledgment; they ensure minimum overhead but don't deliver any guarantees of delivery.
- QoS 1 - At least once: Message re-delivery takes place until the acknowledgment is received, which ensures delivery and causes duplicate messages.
- QoS 2 - Exactly once: The most reliable QoS level where delivery occurs only once using a four-step handshake protocol.

While the above QoS levels increase reliability of the communication, they lack security from any form of unauthorized entry.

B. Security Weaknesses of Standard MQTT

Although highly efficient, MQTT was originally designed for low overheads rather than security. Thus, the default specification of MQTT lacks a considerable number of inherent security measures. In case of implementation of MQTT without any additional safeguards, the following vulnerabilities become possible. Firstly, data sent in plain text can be intercepted by hackers using wireless networks, thereby leaking sensitive data from sensors. Secondly, weak means of authentication can enable hackers to impersonate legitimate devices or to get access to the broker's system. Thirdly, MQTT brokers normally deliver received messages without checking their authenticity, thus becoming an easy target for hackers unless encrypted. An additional vulnerability is connected with replay attacks. In fact, there is no mechanism in MQTT which would ensure the freshness of the messages; thus, if the application does not implement any counters or timestamps, hackers will be able to resend the intercepted data. Indeed, researchers showed that a considerable number of MQTT implementations use default credentials, unsecured wireless networks or no encryption at all. Therefore, a secure MQTT system requires the implementation of the following measures:

- Strong authentication
- Authorization
- Encryption
- Integrity verification
- Replay protection
- Secure broker configuration

C. Authentication Mechanisms

It is important that only authorized devices can communicate with the MQTT broker through authentication. Without the correct authentication process, any unauthorized device can publish or subscribe to the MQTT topic. There are two ways that authentication can be done in MQTT.

1) *Username and Password Authentication*: The most basic form of authentication consists of usernames and passwords set up in the MQTT broker. Clients send their credentials prior to any communication, and once confirmed by the broker, the client gains access. This approach has the following benefits:

- Simplicity of implementation
- Low computational needs
- Good compatibility

Nonetheless, this approach has its drawbacks. In case credentials are sent without TLS encryption, attackers will be able to steal them through packet sniffer programs. Weak passwords will also make this type of attack more likely. Therefore, the researchers advise that usernames/passwords authentication be used along with encrypted communications.

2) *Client Certificates*: Authentication can be improved through the use of digital certificates using the Public Key Infrastructure (PKI). In this method, mutual authentication is ensured by verifying each other's certificate during connection setup. Some of the benefits of using certificate-based authentication include:

- Strong device identity verification
- Resistance to credential theft
- Protection against impersonation attacks
- Improved scalability for enterprise deployments

Even though certificate management is complicated, it is definitely more secure than password security. In the case of constrained embedded devices, the lightweight certificate version is normally chosen due to resource restrictions.

D. Access Control and Authorization

Authentication verifies who a device is, whereas authorization determines what the device is permitted to do. Access control prevents authenticated devices from accessing MQTT topics beyond their intended privileges.

1) *Topic Based Access Control*: Topic-based authorization restricts clients to predefined MQTT topics. For example :

TABLE II
MQTT TOPIC PERMISSIONS FOR SMART HOME DEVICES

Device	Allowed Topic	Permission
ESP32 Door Sensor	home/frontdoor/status	Publish Only
ESP32 Smart Lock	home/frontdoor/status	Subscribe Only
Administrator	home/#	Publish & Subscribe

This approach ensures that compromised devices cannot manipulate unrelated smart-home components.

2) *Broker Hardening Techniques*: Broker hardening refers to configuring the MQTT broker to minimize security vulnerabilities. Recommended practices include:

- Deny any anonymous connection
- Enforce strong passwords
- Enable TLS encryption
- Restrict topic permissions
- Limit simultaneous client connections
- Enable logging and monitoring
- Regularly update broker software

E. Secure MQTT Communication

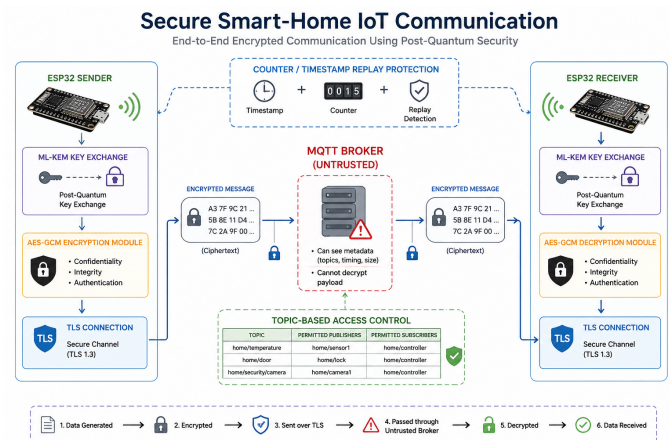
Contemporary MQTT systems often make use of more than one form of security system to achieve confidentiality, integrity, and authentication at the same time. Instead of relying on the

security of the broker alone, it is advised that the cryptographic security is used on the communicating IoT devices themselves. This because there is no trust of the broker in general; this is true for the current project, where the Raspberry Pi broker is just responsible for forwarding the encryption messages. And to mention , Transport Layer Security (TLS) encrypts communication between MQTT clients and the broker. TLS provides:

- Confidentiality
- integrity
- Authentication
- Secure Session Establishment

TABLE III
SECURITY MECHANISMS USED IN MQTT-BASED SMART HOME SYSTEMS

Mechanism	Purpose	Advantages	Limitations
Username & Password	Client authentication	Simple and lightweight	Vulnerable without TLS.
Client Certificates	Mutual authentication	Strong identity verification	Certificate management overhead.
Topic-Based Authorization	Access control	Restricts unauthorized topic access	Requires careful configuration.
TLS	Secure communication channel	Protects data in transit	Broker can still view plaintext.
End-to-End Encryption	Device-to-device confidentiality	Broker cannot decrypt messages	Requires secure key management.
AES-GCM	Confidentiality and integrity	Authenticated encryption in one operation	Requires a secure shared key.
ML-KEM-512	Quantum-safe key exchange	Resistant to quantum attacks	Higher computational cost.



IV. CRYPTOGRAPHIC SOLUTIONS FOR IOT SECURITY

A. Fundamentals of Cryptography in IoT

Cryptography serves as a basis for the security in the Internet of Things communication system by ensuring the protection of sensitive data from any unauthorized use or cyber attack. In light of the fact that IoT devices regularly send data via wireless networks, the application of cryptographic methods ensures that data is kept confidential and cannot be changed. Without proper cryptographic protection, cyber attackers can easily intercept, alter, or even produce fake messages about the smart home system. In contrast to regular computer systems, embedded IoT systems have strict limitations on memory size, computing power, and battery life. As such, any cryptographic method applied in IoT has to strike a proper balance between security and efficiency. IoT security in today's world revolves around four main targets:

- **Confidentiality:** Ensure that no unauthorized user can access the data being transferred.
- **Integrity:** Make sure that the data transferred remains intact throughout the transfer process.
- **Authentication:** Verify the authenticity of communicating devices.
- **Freshness:** Ensure that messages being transferred are new.

This combination of security goals ensures trust between IoT devices.

B. Symmetric Encryption Techniques

In symmetric cryptography, the same private key is used for encryption and decryption purposes. Symmetric algorithms utilize less computational effort than asymmetric algorithms; hence, they are suitable for implementation in microcontroller-based embedded devices like ESP32. Once two IoT devices establish a common secret key in a secure manner, they can use symmetric encryption techniques to encrypt messages sent between each other. Simple mathematical computation is enough for encryption and decryption processes; therefore, high data transfer rates can be achieved. Various symmetric encryption algorithms exist, but AES has emerged as the international standard for embedded security.

1) **AES Encryption:** One such symmetric encryption scheme that is popularly used in current cybersecurity is the Advanced Encryption Standard (AES). This standard was introduced by the National Institute of Standards and Technology (NIST) and replaces the previous DES encryption standard with much better security and high efficiency. It uses fixed block sizes of 128 bits and three different key sizes of:

TABLE IV
AES ENCRYPTION KEY SIZES

AES Version	Key Size
AES-128	128 bits
AES-192	192 bits
AES-256	256 bits

C. Replay Protection Techniques

Despite encryption and authentication being put in place to protect communications, a replay attack is still viable if past ciphertext can be reused. Replay countermeasures guarantee that the message that has been received is both fresh and distinct. Some of the common techniques used are counters and timestamps.

TABLE V
SECURITY GOALS AND COMMON SOLUTIONS

Security Goal	Common Solution
Confidentiality	AES-GCM
Integrity	AES-GCM or HMAC-SHA256
Authentication	Username/Password, Certificates, HMAC
Replay Protection	Counter or Timestamp
Secure Key Exchange	ML-KEM (Kyber)
End-to-End Encryption	AES-GCM

V. POST-QUANTUM CRYPTOGRAPHY FOR IOT

A. Motivation for Post-Quantum Cryptography

One of the biggest threats to the future of cybersecurity is the fast-paced development of quantum computing. Public key cryptography methods such as RSA and ECC utilize certain mathematical problems which cannot be solved efficiently using classical computing resources. Yet quantum computing with Shor's algorithm will be able to solve these mathematical problems easily making many cryptographic methods unusable in the future. While currently practical quantum computing is not available which would make currently used cryptographic methods useless, information which is being transmitted and then decrypted by quantum computers can be saved for years until quantum computing reaches that level. This threat is often called "Harvest Now, Decrypt Later" (HNDL). It has led many government organizations and companies to start moving towards post-quantum cryptographic algorithms. Post-quantum cryptography (PQC) is especially important for smart-home IoT devices which can work for many years without being replaced with new hardware.

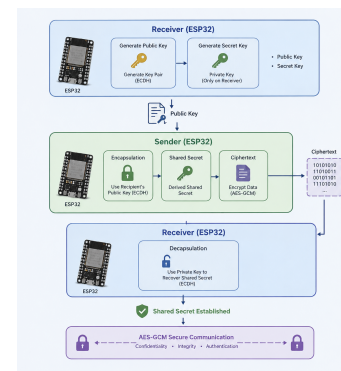


TABLE VI
PROPERTIES OF ML-KEM-512

Property	ML-KEM-512
Security Level	NIST Level 1
Quantum Resistant	Yes
Memory Requirement	Moderate
Computational Complexity	Low-Moderate
Embedded Suitability	Excellent

B. Existing Research on Kyber in IoT Systems

Recent research efforts have become more oriented towards exploring the potential use of ML-KEM in embedded microcontrollers. Currently, most research efforts in this area use ARM Cortex-M processors, ESP32 boards, STM32 microcontrollers, and Raspberry Pi systems. The main goal of such research efforts is to find out if the post-quantum encryption algorithms work under severe memory and processing constraints on embedded IoT devices. From the current state of research, it is clear that ML-KEM-512 is possible to run on embedded devices.

TABLE VII
COMPARISON OF CLASSICAL AND POST-QUANTUM KEY EXCHANGE ALGORITHMS

Algorithm	Security Level	Memory	ESP32
RSA-2048	High (Classical)	High	Poor
ECC P-256	High (Classical)	Medium	Good
ML-KEM-512	Post-Quantum	Medium	Excellent
ML-KEM-768	Post-Quantum	High	Moderate
ML-KEM-1024	Post-Quantum	Very High	Limited

VI. CRYPTOGRAPHY ON RESOURCE-CONSTRAINED DEVICES

A. Characteristics of Embedded IoT Devices

The Internet of Things devices used in an embedded manner are developed to execute some specific tasks with the limitations of hardware and energy. Unlike desktop and server devices, embedded devices use low energy consumption, small size, and low cost as the main criteria and do not care about their computational abilities. Embedded devices can be used for such smart-home purposes as environment monitoring, smart locks, motion detection, lighting control, and home automation. At the same time, embedded devices face some particular cybersecurity problems. The limited computing abilities hinder the implementation of some computational-intensive cryptography algorithms, and the limited memory does not allow large executable files and cryptographic libraries to work. Besides, many IoT devices work permanently by consuming battery or low energy resources. Therefore, they need effective ways of security which will not consume much energy. With the growth of the number of devices, it has become an important field of research to provide cybersecurity of embedded communication. Modern IoT devices should strike a good balance between cryptography safety and acceptable execution time, memory, and energy.

TABLE VIII
ESP32 HARDWARE SPECIFICATIONS

Feature	Specification
CPU	Dual-Core Tensilica LX6 (up to 240 MHz)
SRAM	Approximately 520 KB
Flash Memory	Typically 4 MB (varies by board)
Wireless Connectivity	Wi-Fi 802.11 b/g/n and Bluetooth
Cryptographic Hardware	AES, SHA, RSA, Random Number Generator
Operating Voltage	3.3 V

The ESP32 is one of the most common microcontrollers that are used in embedded Internet of Things systems because of their connectivity capabilities, computing abilities, and affordability. The ESP32 microcontroller is manufactured by Espressif Systems and includes features such as Wi-Fi and Bluetooth connectivity along with computing capabilities for using cryptographic functions.

B. Processing Constraints

Various mathematical calculations are performed during cryptographic computations, which make heavy demands on the processor. Compared to symmetric encryption, public-key algorithms always demand much more computational work. These usually consist of the following:

- Key generation
- Encryption
- Decryption
- Hash generation
- Tag verification

Even though the dual-core processor of the ESP32 enhances computational efficiency, care should be taken in designing cryptographic applications to prevent high communication latency. Luckily, key establishment is an infrequent event.

VII. COMPARATIVE ANALYSIS OF EXISTING RESEARCH

A. Comparison of IoT Security Approaches

In the last ten years, many security proposals were offered to solve the problem of the increasing number of cyber attacks against IoT devices. These proposals vary based on their security goals, computational requirements, implementation difficulty, and compatibility with resource-constrained embedded devices. The first proposals for IoT security were focused on security measures at the network level, namely firewall, VPNs, and secure gateways. Even though these techniques add an extra layer of protection, they do not guarantee security for the end devices in case of attack on the communication channel or broker. That is why the current research trends in this field involve end-to-end security, which means that cryptographic protection can be realized on embedded devices. The second important direction in IoT security research is authentication. Username-password authentication is commonly used because of its computational simplicity; however, it is vulnerable to attacks on credentials, brute force attacks, and weak password management. The next step to improve security is to implement certificate-based authentication, which is more secure but has additional challenges in terms of certification

management. There has been substantial improvement in communication security as well. In previous implementations of MQTT, there was often a tendency to employ only TLS for securing communications between clients and brokers. While TLS is sufficient for protecting the data being transmitted, it does not provide any mechanism for preventing an attacked broker from decrypting the data. Therefore, it is advisable to use TLS along with end-to-end encryption, as it would ensure that only the communicating devices have the keys. In current smart home security architecture, a combination of different security mechanisms such as authenticated encryption, secure key exchange, replay prevention, broker authentication, and access control is used.

TABLE IX
COMPARISON OF SECURITY APPROACHES FOR ESP32-BASED SMART HOME SYSTEMS

Approach	Advantages	Limitations	ESP32
Username & Password	Simple implementation	Weak against credential theft	High
TLS	Encrypts communication channel	Broker can access plaintext	High
End-to-End Encryption	Broker cannot decrypt messages	Requires secure key management	High
AES-GCM	Confidentiality and integrity	Requires shared secret key	Excellent
ECC	Efficient classical key exchange	Vulnerable to quantum attacks	Good
ML-KEM-512	Quantum-resistant key exchange	Larger public keys	Excellent

1) *Challenges of Real-World ESP32 Deployment:* While lab tests prove that sophisticated encryption algorithms run effectively on ESP32 modules, practical usage poses several extra issues. These include:

- Difference in wireless communication.
- Battery capacity constraints.
- Firmware updates.
- Key protection.
- Scalability of devices.
- Latency.
- Physical protection.

The authors stress that for practical application optimization is needed to provide the good user experience and security at the same time. Further research should be conducted on the large-scale smart homes using several communicating devices.

TABLE X
SUMMARY OF RELATED LITERATURE

Paper	Topic	Method	Findings	Limitations
Paper 1	IoT Security	Survey	Identified major IoT threats.	No experimental evaluation.
Paper 2	MQTT Security	Experimental	TLS improves communication security.	Broker remains trusted.
Paper 3	AES-GCM	Performance evaluation	Efficient authenticated encryption.	Requires secure key exchange.
Paper 4	ML-KEM	Embedded implementation	Practical on ESP32.	Increased memory usage.
Paper 5	Post-Quantum IoT	Comparative study	ML-KEM-512 suitable for IoT.	Limited large-scale testing.

VIII. CONCLUSION

This literature review considers current studies dealing with cybersecurity problems in the area of IoT smart homes and, specifically, secure communication between the embedded devices. The review revealed that eavesdropping, spoofing, replay attack, message tampering, denial-of-service attack, malware, and physical attack were the most widespread types of attacks occurring in the IoT environment. It was shown that lightweight communication protocols like MQTT are highly efficient regarding resource-limited devices but require some security measures in order to secure information during transmission. Authentication, authorization, encryption of the communication, and replay attack protection are recommended by all researchers. Moreover, the review shows the necessity of using post-quantum cryptography since the development of quantum computers threatens the traditional public-key encryption algorithms. Among the post-quantum cryptographic schemes already adopted as standards, the ML-KEM-512 proved to be the best key encapsulation algorithm for embedded IoT devices because of its security properties and low computation and memory footprint. Lastly, based on the performance results provided in literature, it can be concluded that ESP32 microcontroller is capable of handling both AES-GCM and optimized ML-KEM-512 efficiently enough to support secure smart-home communication.